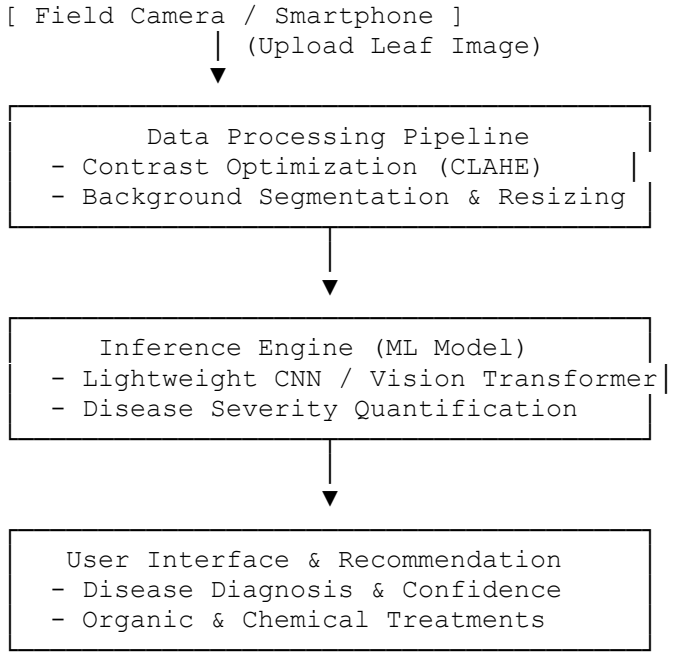
# Develop a Machine Learning solution for crop disease detection

B.AMAR-192421397

## 1. System Architecture & Workflow

A production-ready pipeline consists of four distinct phases:




## 2. Dataset Strategy

To ensure model robustness across varied lighting and field conditions, combine laboratory-grade baseline datasets with real-world, in-field image collections:

- **Primary Datasets:**
  - **PlantVillage:** Benchmarking dataset (54,303 lab-captured leaf images across 38 classes).
  - **PlantDoc:** Real-world field images captured under variable lighting, dirt, and occlusions.
  - **Rice/Tomato Disease Datasets:** Specific field datasets available on Kaggle/Roboflow for targeted regional deployment.
- **Data Augmentation Strategy:**
  - **Geometric:** Random rotation ($\pm 30^\circ$), horizontal/vertical flips, affine transformations.
  - **Photometric:** Random brightness adjust ($\pm 20\%$), Gaussian blur, Contrast Limited Adaptive Histogram Equalization (CLAHE).

- **Domain-Specific:** CutMix and MixUp to prevent overfitting on clean background features.

# 3. Model Architecture Selection

Depending on deployment requirements, select an architecture that balances latency and accuracy:

| Strategy | Recommended Models | Pros | Cons |
|---|---|---|---|
| **Edge / Mobile Deployment** | MobileNetV3-Large, EfficientNet-B0 | Ultra-fast latency (<50ms), small footprint (<20MB) | Slightly lower accuracy on subtle early-stage lesions |
| **High Accuracy / Cloud Server** | ConvNeXt-Base, Swin Transformer, ResNet-50 | High feature resolution, robust against noisy field backgrounds | Slower inference speed, requires cloud deployment |

# 4. End-to-End Implementation (PyTorch)

This production-grade script covers data augmentation, transfer learning using `MobileNetV3`, and automated feature extraction:

Python

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader

# 1. Define Production Augmentation & Preprocessing Pipeline
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(25),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

val_transforms = transforms.Compose([
    transforms.Resize(256),
```

```
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

# 2. Dataset Loading (Assuming folder layout: root/class_name/image.jpg)
# train_dataset = datasets.ImageFolder('data/train',
transform=train_transforms)
# val_dataset = datasets.ImageFolder('data/val', transform=val_transforms)

# 3. Model Architecture Construction (Transfer Learning)
def build_crop_disease_model(num_classes: int, pretrained: bool = True):
    # MobileNetV3 Large balance speed and accuracy for edge devices
    weights = models.MobileNet_V3_Large_Weights.DEFAULT if pretrained else
None
    model = models.mobilenet_v3_large(weights=weights)

    # Freeze feature extraction backbone
    for param in model.features.parameters():
        param.requires_grad = False

    # Replace classification head
    in_features = model.classifier[3].in_features
    model.classifier[3] = nn.Sequential(
        nn.Dropout(p=0.3),
        nn.Linear(in_features, num_classes)
    )
    return model

# 4. Training Loop setup
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
num_disease_classes = 38  # Example: 38 PlantVillage classes
model =
build_crop_disease_model(num_classes=num_disease_classes).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.classifier.parameters(), lr=1e-3,
weight_decay=1e-4)

print(f"Model initialized on device: {device}")
```

# 5. Model Explainability & Diagnostics (Grad-CAM)

To ensure the model is evaluating the **actual leaf lesion** (and not learning background biases like dirt or shadows), implement **Grad-CAM (Gradient-weighted Class Activation Mapping)**:

**Why it matters:** If the activation map highlights soil or background noise instead of yellowing/spots on the leaf blade, the model is overfitting to dataset artifacts and will fail in real field scenarios.

Python

```
# Conceptual Grad-CAM integration snippet
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image

# Target the final convolutional layer of MobileNetV3
```

```
target_layers = [model.features[-1]]
cam = GradCAM(model=model, target_layers=target_layers)

# Generates heatmap overlay on top of input leaf tensor
# grayscale_cam = cam(input_tensor=input_tensor, targets=targets)
```

# 6. Real-World Deployment Strategy

1. **Quantization & Edge Optimization:**
    - Convert the PyTorch model to **ONNX** format, then apply **INT8 Quantization** via TensorFlow Lite or TensorRT.
    - Decreases model size from ~40MB to **~10MB** with minimal impact on accuracy, enabling offline inference inside mobile apps (e.g., Flutter + TFLite runtime).
2. **API Backend:**
    - Build a lightweight **FastAPI** wrapper that accepts base64-encoded leaf photos, crops the leaf using a YOLO v8 detector, runs the classification model, and returns a JSON response:

JSON

```
{
  "crop": "Tomato",
  "disease": "Early Blight",
  "confidence": 0.942,
  "severity_index": "Moderate (15-20% leaf surface)",
  "treatment": [
    "Apply copper-based fungicide.",
    "Remove infected lower leaves to prevent spore splash."
  ]
}
```

**1.Model Optimization:**Post-training setup.

Export trained PyTorch parameters to ONNX format and perform INT8 dynamic quantization.

**2.Edge Integration:**Mobile App / Web App.

Bundle quantized model into a Flutter or React Native mobile framework for offline field inference.

**3.API & Treatment Engine:**Backend Services.

Connect predicted class label to an automated database mapping disease keys to local agricultural remedies.